GNNExplainer: Generating Explanations for Graph Neural Networks

Arion Scheid
Department of Computer Science
TU Wien
Vienna, Austria

Abstract

Graph Neural Networks (GNNs) achieve strong performance on graph-structured data but are largely opaque: it is unclear which nodes, edges, or features drive a given prediction. This paper examines GNNEXPLAINER [Ying et al., 2019], a post-hoc method that finds the compact subgraph and feature subset that maximize mutual information with a trained model's prediction. We present a self-contained derivation of this objective and its continuous relaxation into learned edge and feature masks, including a step we show is mis-stated in the original derivation, connecting it to the practical optimization implemented in PyTorch Geometric [Fey and Lenssen, 2019]. We study two failure modes. First, gradient-based mask optimization can become trapped in local optima; on a novel multi-size clique dataset, explanation F1 stays between 0.13 and 0.26 across clique sizes and the optimizer never recovers a full clique. Second, motivated by 2026 work showing that jointly-trained *self-explainable* GNNs can produce explanations provably unrelated to a model's actual reasoning [Azzolin et al., 2026], we add two constant, label-independent color features to the clique dataset and test whether GNNEXPLAINER's feature mask mistakes either one for a real signal. We find a partial version of this failure: one color feature shows almost no class-dependent gap, but the other shows a large one, disproportionate to the underlying difference in classifier accuracy between classes.

1 Introduction

Graph Neural Networks have become the standard approach for learning on graph-structured data, with applications in molecular property prediction, fraud detection, and scientific knowledge graphs. Despite their success, GNNs are black boxes: a prediction cannot easily be traced to specific input elements. This is not merely an aesthetic concern, since a predicted toxicity must be linked to a functional group to guide synthesis, and a flagged transaction must be auditable for regulators.

The general problem of explaining black-box predictions has been studied extensively [Lipton, 2018], but standard tools such as LIME [Ribeiro et al., 2016] and SHAP [Lundberg and Lee, 2017] treat inputs as flat feature vectors and cannot account for graphs' relational structure, so a dedicated method is needed.

GNNEXPLAINER [Ying et al., 2019] fills this gap. Given any trained GNN and a node to explain, it identifies the minimal subgraph and node feature subset that best account for the prediction, by maximizing *mutual information* with the model's output: a good explanation, fed to the frozen GNN alone, reproduces the original prediction as confidently as possible. GNNEXPLAINER operationalizes this with a continuous, differentiable *mask* over the edges and features of the local computation graph, optimized by gradient descent. The method is post-hoc and model-agnostic: it works on any GNN without retraining, and the GNN's parameters stay *frozen* throughout.

This paper makes four contributions. First, we derive the mutual information objective from scratch, identifying a step in the original derivation where Jensen's inequality is applied in the wrong direction (Section 4.5). Second, we concretely investigate the local-optima behavior that Ying et al. [2019] already anticipate but do not characterize: when it settles on sub-cliques or sub-motifs that are locally but not globally optimal. Third, we connect GNNEXPLAINER to 2026 work on *self-explainable* GNNs showing that some explanation mechanisms can be provably unrelated to a model's actual reasoning, adapting this construction into a testable hypothesis for GNNEXPLAINER. Fourth, we implement GNNEXPLAINER on a multi-size clique benchmark with known ground-truth explanations, demonstrating the local-optima behavior and empirically testing whether the unfaithfulness failure above also affects GNNEXPLAINER's feature mask.

2 Related Work

Graph Neural Networks. The message-passing framework of Gilmer et al. [2017] unifies most GNN architectures. Examples of such are GCN (Graph Convolutional Networks) of Kipf and Welling [2017] which use symmetric normalized aggregation or GAT (Graph Attention Networks) of Veličković et al. [2018] which use learned attention weights, sometimes interpreted as explanations (as in Ying et al. [2019]), though Jain and Wallace [2019] show high attention on an edge does not imply that edge is necessary for the prediction.

GNN Explanation Methods. A comprehensive taxonomy of GNN explanation methods is provided by Yuan et al. [2023]. Among instance-level methods, GNNEXPLAINER [Ying et al., 2019] is the seminal, most widely cited method and the primary reference for this paper. Li and Saude [2020] extend the idea to weighted graphs. PGExplainer [Luo et al., 2020] addresses GNNEXPLAINER's consistency limitation with a shared generative model instead of a separate optimization per instance. SubgraphX [Yuan et al., 2021] avoids gradient-based optimization entirely, searching combinatorially instead. XGNN [Yuan et al., 2020] trains a graph generator at the model level.

Evaluation of GNN Explanations. A substantial literature asks not how to produce GNN explanations but whether to trust them. Faber et al. [2021], on evaluation methodology, identify five pitfalls in comparing explanations to planted ground truth (Section 7), and Agarwal et al. [2022] give the first theoretical reliability analysis, bounding faithfulness, stability, and fairness. This report sits in the same field: Section 6 applies both papers' cautions to our results, and Section 4.8 extends the same question to a construction from Azzolin et al. [2026], discussed next.

Self-Explainable GNNs and Unfaithful Explanations. Azzolin et al. [2026] study this question for *self-explainable* GNNs (SE-GNNs). An SE-GNN couples an *explanation extractor* e, mapping an input graph to a subgraph $e(G) = R \subseteq G$, with a *classifier* g that predicts from R alone, so the full model is $f(G) = g(e(G))$; GSAT [Miao et al., 2022] is a representative instance. Since e and g train jointly, e can discard the information g needs and instead report a recurrent, non-discriminative pattern: an *anchor set* $Z = \{z_y\}_{y \in \mathcal{Y}}$ of single-node subgraphs, one per class, with $z_y \subseteq G$ in *every* instance G (Appendix C gives a worked example). High accuracy therefore cannot certify that an SE-GNN's explanation reflects its real computation. Section 4.8 asks whether a weaker analogue of this result survives for post-hoc explainers such as GNNEXPLAINER.

3 Background: Graph Neural Networks

A graph is a tuple $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{X})$ with node set $\mathcal{V}$ of size $n = |\mathcal{V}|$, edge set $\mathcal{E}$, and node feature matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$, where d is the number of features per node. The adjacency matrix $\mathbf{A} \in \{0,1\}^{n \times n}$ encodes the edge set. We assume undirected graphs throughout and write $\mathcal{N}(v) = \{u \in \mathcal{V} : (v,u) \in \mathcal{E}\}$ for the neighbors of node v.

GNNs are trained for a classification objective: given $\mathcal{G}$, predict a label from a fixed set of C classes $\{1, \ldots, C\}$. This prediction can be made at several levels. Node classification predicts a true label $y_v \in \{1, \ldots, C\}$ for every node $v \in \mathcal{V}$. Link prediction predicts a label for every pair of nodes. Graph classification predicts a single label for the whole graph. We focus on node classification throughout, following Kipf and Welling [2017].

A K-layer GNN, which we denote Φ, computes node embeddings by iterating the message-passing scheme of Gilmer et al. [2017]. At layer k, each node v first aggregates information from its neighbors and then updates its own representation:

$$\mathbf{m}_v^{(k)} = \mathrm{AGG}^{(k)}\big(\{\phi^{(k)}(\mathbf{H}_v^{(k-1)}, \mathbf{H}_u^{(k-1)}, \mathbf{A}_{vu}) : u \in \mathcal{N}(v)\}\big), \tag{1}$$

$$\mathbf{H}_v^{(k)} = \mathrm{UPDATE}^{(k)}\big(\mathbf{H}_v^{(k-1)}, \mathbf{m}_v^{(k)}\big), \tag{2}$$

with initialization $\mathbf{H}_v^{(0)} = \mathbf{X}_v$, the raw node features. Here $\mathbf{H}_v^{(k)} \in \mathbb{R}^{d_k}$ is the *hidden representation* (embedding) of node v after k layers. $\mathbf{m}_v^{(k)}$ is the *aggregated message* received from v's neighbors at layer k. $\phi^{(k)}$ is a learnable function, for example a linear transformation, that combines a pair of node embeddings with the edge weight $\mathbf{A}_{vu}$. $\mathrm{AGG}^{(k)}$ is a permutation-invariant aggregation function over the neighbor set, for example a sum or mean. $\mathrm{UPDATE}^{(k)}$ combines the current node embedding $\mathbf{H}_v^{(k-1)}$ with the incoming message $\mathbf{m}_v^{(k)}$, typically with a linear layer followed by a non-linearity.

The final embedding $\mathbf{H}_v^{(K)}$ is passed through a softmax layer that outputs a probability $P_\Phi(Y = c \mid \mathcal{G}, \mathbf{X})$ for each class c, where Y is the random variable representing the label Φ predicts for v. The predicted label is the most probable class,

$$\hat{y}_v = \operatorname*{arg\,max}_{c \in \{1,\ldots,C\}} P_\Phi(Y = c \mid \mathcal{G}, \mathbf{X}). \tag{3}$$

A critical structural property is that $\hat{y}_v$ depends only on the K-hop neighborhood of v. We call this neighborhood the *computation graph* $\mathcal{G}_c^v$, with associated features $\mathbf{X}_c^v$. All nodes and edges outside $\mathcal{G}_c^v$ are irrelevant to $\hat{y}_v$, so $P_\Phi(Y = c \mid \mathcal{G}, \mathbf{X}) = P_\Phi(Y = c \mid \mathcal{G}_c^v, \mathbf{X}_c^v)$: only $\mathcal{G}_c^v$ matters for explaining $\hat{y}_v$.

4 GNNExplainer

4.1 Problem Formulation

The GNN Φ of Section 3 is trained once and then kept *frozen* during explanation. The goal is to identify the part of the computation graph $\mathcal{G}_c^v$ that best explains the prediction $\hat{y}_v$.

Problem 1 (Instance-Level GNN Explanation [Ying et al., 2019])**.** Given Φ, a computation graph $\mathcal{G}_c^v$ with features $\mathbf{X}_c^v$, and prediction $\hat{y}_v$, find a subgraph $\mathcal{G}_S \subseteq \mathcal{G}_c^v$ and a masked feature matrix $\mathbf{X}_S^F$ such that $(\mathcal{G}_S, \mathbf{X}_S^F)$ is most *informative* about $\hat{y}_v$, i.e., knowing $(\mathcal{G}_S, \mathbf{X}_S^F)$ reduces uncertainty about $\hat{y}_v$ as much as possible.

4.2 Information-Theoretic Preliminaries

GNNExplainer formalizes "most informative" using *mutual information*. We first define the necessary information-theoretic quantities for the random variable Y of Eq. (3), which ranges over the C classes $\{1, \ldots, C\}$ of Section 3.

The *Shannon entropy* measures the uncertainty of Y:

$$H(Y) = -\sum_{c=1}^{C} P(Y = c)\, \log P(Y = c). \tag{4}$$

$H(Y) \geq 0$, with equality only when one class has probability 1 (no uncertainty), and is maximized when all classes are equally likely. The *conditional entropy* measures the residual uncertainty in Y after observing evidence Z:

$$H(Y \mid Z) = -\sum_{c=1}^{C} P(Y = c \mid Z)\, \log P(Y = c \mid Z), \tag{5}$$

with $H(Y \mid Z) \leq H(Y)$: conditioning on informative evidence can only reduce uncertainty. *Mutual information* quantifies this reduction:

$$\mathrm{MI}(Y; Z) = H(Y) - H(Y \mid Z) \;\geq\; 0. \tag{6}$$

A high value of $\mathrm{MI}(Y; Z)$ means observing Z makes Y substantially more predictable. For example, a confident 3-class prediction $P = (0.9, 0.05, 0.05)$ has $H \approx 0.47$ bits, while a near-uniform prediction $P = (0.33, 0.33, 0.34)$ has $H \approx 1.10$ bits.

4.3 Mutual Information Objective

GNNEXPLAINER seeks the explanation $(\mathcal{G}_S, \mathbf{X}_S^F)$ that maximizes the mutual information between Y and this evidence:

$$\max_{\mathcal{G}_S \subseteq \mathcal{G}_c^v} \mathrm{MI}\big(Y;\, (\mathcal{G}_S, \mathbf{X}_S^F)\big) = \underbrace{H(Y)}_{\text{constant}} - H\big(Y \mid \mathcal{G}=\mathcal{G}_S,\, \mathbf{X}=\mathbf{X}_S^F\big). \tag{7}$$

Since Φ is fixed, $H(Y)$ is a constant. Maximizing MI is therefore equivalent to *minimizing the conditional entropy*, that is, finding the subgraph that keeps the GNN most confident:

$$\min_{\mathcal{G}_S,\, \mathbf{X}_S^F} H\big(Y \mid \mathcal{G}=\mathcal{G}_S,\, \mathbf{X}=\mathbf{X}_S^F\big) = -\sum_{c=1}^{C} P_\Phi(Y=c \mid \mathcal{G}_S, \mathbf{X}_S^F)\, \log P_\Phi(Y=c \mid \mathcal{G}_S, \mathbf{X}_S^F). \tag{8}$$

This uses the **entire softmax distribution** over all C classes and penalizes any residual uncertainty.

Class-specific formulation. In practice, the GNN's predicted label $\hat{y}_v = c^*$ is known. Ying et al. [2019] replace the full conditional entropy with a *cross-entropy targeted at* c^*:

$$\min_{\mathcal{G}_S,\, \mathbf{X}_S^F} -\log P_\Phi\big(Y=c^* \mid \mathcal{G}=\mathcal{G}_S,\, \mathbf{X}=\mathbf{X}_S^F\big). \tag{9}$$

While Eq. (8) asks *"Is the GNN confident?"* across all classes, Eq. (9) asks *"Does $\mathcal{G}_S$ support predicting c^*?"* by only looking at $P(Y=c^*)$. The two agree when the GNN is highly confident. Eq. (9) is simpler, directly interpretable as the standard cross-entropy loss used in classification, and is the form implemented in PyTorch Geometric.

4.4 Mask Parameterization

Searching over all $2^{|\mathcal{E}_c^v|}$ subgraphs by brute force is intractable. GNNEXPLAINER sidesteps this with continuous *masks*: real-valued scalars (one per edge or feature dimension) passed through a sigmoid to produce soft weights in $(0, 1)$, optimized jointly by gradient descent, with the final discrete subgraph recovered by thresholding.

Edge mask. An *edge mask* is a real-valued matrix $\mathbf{M}_A \in \mathbb{R}^{n \times n}$ learned with one entry per edge in $\mathcal{G}_c^v$. Applying the sigmoid $\sigma(\cdot)$ gives a soft weight in $(0, 1)$, and the masked adjacency is:

$$\mathbf{A}_S[i,j] = \mathbf{A}[i,j] \cdot \sigma\big(\mathbf{M}_A[i,j]\big), \quad (i,j) \in \mathcal{E}_c^v. \tag{10}$$

This soft-masked adjacency replaces the binary one in all GNN message-passing computations, maintaining differentiability. In practice, the objective in Eq. (9) is evaluated by a single forward pass through the soft-weighted graph. This is a mean-field approximation, discussed in Section 7.

Feature mask. Ying et al. [2019] intend the feature mask $F \in \{0,1\}^d$ as a *binary* selector, $\mathbf{x}_j^F = \mathbf{x}_j \odot F$. They warn against naively relaxing F to a continuous $\sigma(\mathbf{m}_F) \in (0,1)^d$ multiplied directly into $\mathbf{x}_j$, as done for the edge mask (Eq. 10): an important feature whose value is already close to zero becomes indistinguishable from an unimportant one, since scaling it near zero either way gives almost no gradient signal about its true importance. Their proposed fix, a marginal-distribution reparametrization trick, is summarized in Appendix B.

The PyTorch Geometric implementation used in our experiments (`node_mask_type='attributes'`) skips this reparametrization entirely. It learns an unconstrained mask $\mathbf{M}_F \in \mathbb{R}^{n \times d}$ (one entry per node *and* feature, not shared across nodes as originally proposed) and applies it exactly like the edge mask,

$$\mathbf{X}_S^F = \mathbf{X}_c^v \odot \sigma(\mathbf{M}_F), \tag{11}$$

i.e., the same near-zero-value blind spot is present in the library. A practitioner explaining informative small-valued features should be aware GNNEXPLAINER may under-report their importance; every node feature in our experiments is a $0/1$ indicator, however, so this case does not arise and Eq. (11) is adequate for our purposes.

4.5 From a Discrete Search to a Single Forward Pass: A Flawed Step

Eqs. (8)–(9) still require searching over discrete subgraphs $\mathcal{G}_S$. Ying et al. [2019] make this tractable by treating $\mathcal{G}_S \sim \mathcal{Q}$ as a random variable drawn from a distribution $\mathcal{Q}$ over subgraphs, and by minimizing $\mathbb{E}_{\mathcal{G}_S \sim \mathcal{Q}}\big[H(Y \mid \mathcal{G}_S, \mathbf{X}_S^F)\big]$. They then invoke *Jensen's inequality* which states that for a convex[1] φ and a random variable Z taking values in its domain, $\varphi(\mathbb{E}[Z]) \leq \mathbb{E}[\varphi(Z)]$. Here $\varphi = H(Y \mid \cdot, \mathbf{X}_S^F)$ is regarded as a function of the fractional adjacency matrix $\mathbf{A}_S$ of Eq. (10). Ying et al. use this to replace the expectation above with $H\big(Y \mid \mathbb{E}_{\mathcal{Q}}[\mathcal{G}_S], \mathbf{X}_S^F\big)$, licensing the single soft-masked forward pass of Eq. (10) in place of an intractable average; they state that, "with convexity assumption," this substitution is an *upper* bound on the original expectation.

This is backwards. Under the very convexity assumption invoked (that $H(Y \mid \mathcal{G}_S, \cdot)$ is convex in $\mathcal{G}_S$), Jensen's inequality actually gives $H(Y \mid \mathbb{E}_{\mathcal{Q}}[\mathcal{G}_S], \mathbf{X}_S^F) \leq \mathbb{E}_{\mathcal{Q}}\big[H(Y \mid \mathcal{G}_S, \mathbf{X}_S^F)\big]$: a *lower* bound, not an upper one, and minimizing a lower bound gives no guarantee about the quantity of actual interest. Worse, the authors immediately note that "due to the complexity of neural networks, the convexity assumption does not hold" for the models GNNEXPLAINER targets, so even the mislabeled bound lacks a valid premise. This step has no rigorous justification; we adopt it only because it is what every practical implementation optimizes, and because its value rests on strong empirical results (Section 6), not on the derivation.

4.6 Optimization Objective

Substituting the mask-parameterized adjacency (Eq. 10) and features (Eq. 11) into the class-specific formulation (Eq. 9), the full objective adds sparsity and discreteness regularization:

$$\min_{\mathbf{M}_A, \mathbf{M}_F} \; -\log P_\Phi(\hat{y}_v \mid \mathbf{A}_S, \mathbf{X}_S^F) + \alpha_1 \sum_{(i,j)} \sigma(\mathbf{M}_A[i,j]) + \alpha_2 \sum_{(i,j)} H\big(\sigma(\mathbf{M}_A[i,j])\big) + \alpha_3 \sum_{v,k} \sigma(\mathbf{M}_F[v,k]) + \alpha_4 \sum_{v,k} H\big(\sigma(\mathbf{M}_F[v,k])\big), \tag{12}$$

where $H(p) = -p \log p - (1-p)\log(1-p)$. The ℓ_1 terms (α_1, α_3) enforce sparsity; the binary-entropy terms (α_2, α_4) push mask values toward 0 or 1. The final subgraph is obtained by thresholding the edge mask at 0.5 or retaining the top-K edges.

4.7 Local Optima in the Mask Optimization

Because GNNEXPLAINER solves Eq. (12) by gradient descent, explanation quality depends on which local optimum the optimizer reaches. Ying et al. [2019] already anticipate this: since their objective is non-convex (Section 4.5), gradient descent finds only *some* local minimum, which they report empirically tends to be a high-quality explanation. This is therefore not a new discovery; our contribution is a concrete investigation of when it instead settles on a poor one, particularly when every strict sub-structure of the correct explanation already maintains reasonable *prediction confidence*, the probability assigned to the predicted class.

Consider a node v in a k-clique in a sparse graph, where the GNN assigns high confidence to class 1 whenever the local neighborhood is dense. A triangle (3-clique) embedded within the k-clique is already denser than the Erdős-Rényi [Erdős and Rényi, 1959] background, so the triangle alone may already preserve confident class-1 prediction. The gradient of Eq. (12) is then small: removing a triangle edge would decrease confidence, and the entropy regularization discourages adding more. The optimizer stops at this *sub-clique local minimum* even though the true explanation is the full k-clique, and as k grows, the number of competing sub-clique local optima grows exponentially. A weak classifier compounds this: if the GNN does not reliably predict clique nodes, the gradient signal is small throughout, and convergence to any meaningful explanation becomes unlikely.

[1] A function φ is *convex* when $\varphi(\lambda a + (1-\lambda)b) \leq \lambda\varphi(a) + (1-\lambda)\varphi(b)$ for all a, b in its domain and $\lambda \in [0,1]$.

4.8 Anchor Features and Unfaithful Explanations in GNNEXPLAINER

The unfaithfulness result reviewed in Section 2 concerns SE-GNNs, where e and g are optimized jointly. GNNEXPLAINER has no such freedom, so we state the mechanism precisely for our setting, to fix what *would* constitute the same failure here and motivate a testable, weaker analogue.

Azzolin et al. [2026] study graph classification, where anchor *nodes* recur across every graph. A set of anchor nodes need not lie inside a given node's computation graph $\mathcal{G}_c^v$ (Section 3), so it cannot serve as a node-level anchor in general, unlike a node *feature*, which is present at every node. This is why we use color, not designated nodes, below.

Augment the multi-size clique dataset with two constant *color* features per node, $\text{color}_1 \equiv 1$ and $\text{color}_2 \equiv 2$, taking the exact same value on *every* node. Both are anchor features (Section 2): $\text{color}_1 = 1$ and $\text{color}_2 = 2$ hold for every node $v \in \mathcal{V}$ regardless of its true label $y_v \in \{0, 1\}$, so neither carries any information about clique membership. A hypothetical SE-GNN pair (e, g) could still reach perfect accuracy while reporting only one of these two, always-true facts as its explanation. Let $e(v) := \{v, \text{color}_1\}$ whenever v truly belongs to a clique, determined by e examining the *entire* graph internally, and $e(v) := \{v, \text{color}_2\}$ otherwise. With $g(\text{color}_1) = \text{clique}$ and $g(\text{color}_2) = \text{non-clique}$, $f = g \circ e$ gives $f(G) = g(e(G)) = g(\text{color}_1) = \text{clique}$ for every clique node. This is an instance of Theorem 1 of Azzolin et al. [2026]: f is perfectly accurate, yet its reported explanation, identically true for every node in the graph, is entirely unrelated to the true decision-relevant structure, visible only inside e's hidden computation and never revealed.

GNNEXPLAINER cannot build this encoding, since Φ is fixed and there is no e to optimize: this is exactly what separates a post-hoc explainer from an SE-GNN. In Theorem 1, the failure lives in the *joint training* of (e, g), so it occurs regardless of how Φ treats the anchor set; a post-hoc explainer can only report what an already-trained Φ does. Any anchor-style failure here must therefore already exist inside Φ itself, whether the anchor is a set of graph-level nodes [Azzolin et al., 2026] or our node-level features: an ideally-trained Φ would ignore them, and GNNEXPLAINER would then ignore them too. If Φ's training instead assigns any non-zero, spuriously small weight to color_1 or color_2, GNNEXPLAINER's feature mask has a real gradient to exploit, and the mean-field optimizer, already prone to local optima (Section 4.7), may inflate that weight into an unfaithful explanation. We test this next.

5 Experiments

5.1 Synthetic Dataset

Multi-Size Clique Detection with a Color Anchor. To study local optima, we embed cliques of sizes 3, 4, and 5 in a single sparse Erdős-Rényi graph [Erdős and Rényi, 1959] with $n = 200$ nodes, each pair independently connected with probability $p = 0.02$: 8 triangles, 6 four-cliques, and 5 five-cliques, covering 73 of the 200 nodes. Labels are binary (0 = non-clique, 1 = clique node). Node features are two constant *color* features, $\text{color}_1 \equiv 1$ and $\text{color}_2 \equiv 2$, with the same value on every node. These are the anchor features of Section 4.8, uninformative about the label by construction. The ground-truth explanation for a node in a k-clique consists of the $\binom{k}{2}$ undirected edges of that clique.

This setting exposes the local-optima behavior of Section 4.7: a triangle embedded in a 5-clique is already a locally consistent explanation, yet represents only 3 of the 10 true explanatory edges.

5.2 GNN Architecture and Training

We train a 2-layer GCN [Kipf and Welling, 2017] with 64 hidden units, ReLU activations, and dropout ($p = 0.5$), for 600 epochs using Adam [Kingma and Ba, 2015] with learning rate 10^{-2} and weight decay 5×10^{-5}. We apply class-weighted cross-entropy ($w_1 = n_{\text{base}}/n_{\text{clique}}$) to compensate for the 127:73 class imbalance. These hyperparameters were chosen to keep the trained classifier reasonably balanced across both classes, since a poorly-tuned network risks under-fitting the base class entirely and confounding the explanation results that follow (Section 7).

Table 1: Explanation accuracy on the clique benchmark. Edges are thresholded at mask weight > 0.5. F1 values are mean $\pm$ std over all explained nodes. "Expl. clique" reports the average size of the largest clique recovered among the true clique nodes in the explanation subgraph.

Clique size	Precision	Recall	F1	Expl. clique
$k = 3$ (triangles)	0.202	0.326	0.199 ± 0.134	1.96 / 3
$k = 4$	0.286	0.329	0.261 ± 0.124	2.56 / 4
$k = 5$	0.156	0.117	0.132 ± 0.110	1.87 / 5
All clique nodes	0.216	0.272	0.201 ± 0.134	–

5.3 Explainer Configuration and Evaluation Metrics

We use the GNNEXPLAINER implementation in PyTorch Geometric [Fey and Lenssen, 2019], run for 150 gradient-descent steps per node at learning rate 10^{-2}, learning both edge and node-feature attribute masks.

Classification accuracy. We first verify the GNN itself by reporting *accuracy*: the fraction of correctly classified nodes, $\mathrm{Acc} = |\{v : \hat{y}_v = y_v\}|/|\mathcal{V}|$.

Explanation quality. Edges with mask weight $\sigma(\mathbf{M}_A[i, j]) > 0.5$ are *selected*; the rest are discarded. Let S be the selected edge set and T the ground-truth edge set for a given node. We report precision, recall, and their harmonic mean F1:

$$P = \frac{|S \cap T|}{|S|}, \qquad R = \frac{|S \cap T|}{|T|}, \qquad F_1 = \frac{2PR}{P + R}. \tag{13}$$

Azzolin et al. [2026] review alternative faithfulness metrics that probe an explanation by perturbing the graph rather than comparing to ground truth (e.g., Fidelity, or their own EST metric), and Faber et al. [2021] and Agarwal et al. [2022] argue such comparison can itself be misleading (Section 7). We use F1 against the known planted ground truth anyway, as it remains the most widely reported metric on synthetic benchmarks with known explanations.

We additionally report the *explanation clique size*: the largest clique found among the true clique nodes in the thresholded explanation subgraph (via `networkx.find_cliques`).

Anchor-feature (color) weight. GNNEXPLAINER optimizes a separate feature mask for each explained node; write $\mathbf{M}_F^{(v)}$ for the mask learned while explaining v (Eq. 11), one row per node in v's computation graph. Let $\mathbf{w}_v \in (0, 1)^2$ be the sigmoid of v's row of $\mathbf{M}_F^{(v)}$: the weight GNNEXPLAINER assigns to v's color_1 and color_2 entries. A faithful explainer should assign $\mathbf{w}_v$ a low value that does not depend on v's class. A class-dependent asymmetry, e.g., a high color_1 weight on clique nodes paired with a high color_2 weight on non-clique nodes, would be a soft version of the deterministic encoding constructed in Section 4.8, though Section 7 argues this would indict Φ's training, not GNNEXPLAINER.

6 Results and Discussion

6.1 Multi-Size Clique Detection

The GCN reaches 78.5% overall accuracy: 79.5% on base nodes and 76.7% on clique nodes, a balanced fit that gives the explainer a real gradient signal on both classes.

Table 1 shows explanation quality still falls well short of full recovery at every clique size, though not strictly monotonically: F1 is 0.199 ($k = 3$), 0.261 ($k = 4$), and 0.132 ($k = 5$), and the optimizer never recovers the full clique on average, at best 2.56 of 4 edges. The $k = 5$ case remains clearly the worst, consistent with the local-optima mechanism of Section 4.7: as k grows, more competing sub-clique local optima exist. The non-monotonic gap between $k = 3$ and $k = 4$ likely reflects the small number of cliques per size (8 and 6) rather than the effect's absence.

6.2 Anchor-Feature (Color) Results

We measure the mean color-feature weight $\mathbf{w}_v$ (Section 5) over 57 explained clique nodes and 50 explained non-clique nodes. A faithful explainer would assign both color_1 and color_2 a weight close to 0, equally for both classes, since both are constant and carry no information about y_v. Neither is suppressed to near zero: clique nodes receive $\text{color}_1 = 0.230$ and $\text{color}_2 = 0.572$, and non-clique nodes receive $\text{color}_1 = 0.209$ and $\text{color}_2 = 0.291$, so the default regularization does not drive a genuinely irrelevant feature to 0 within 150 steps.

The class-dependent gap is uneven across the two features: color_1 still shows only a small gap (0.230 vs. 0.209), but color_2 shows a substantial one (0.572 vs. 0.291), with clique nodes, the very slightly *harder* class here (76.7% vs. 79.5% accuracy), receiving the much higher weight. This direction fits a compensation account, though a 2.8-point accuracy gap cannot explain a 0.281 weight gap on its own. GNNEXPLAINER does not fabricate a clean, deterministic encoding ($g(\text{color}_1)$=clique): 0.572 is still far from 1. But this large, disproportionate gap on color_2 is a clearer instance of the Section 4.8 failure on a reasonably well-balanced classifier, a gap Section 7 attributes to Φ's training, not GNNEXPLAINER.

7 Limitations

Three further limitations qualify our results. First, the mean-field relaxation (Section 4.5) evaluates a soft-masked graph outside the binary-adjacency distribution Φ was trained on: Φ never saw an edge held at half-strength, so its behavior there is unconstrained and may not match how it would behave if that edge were genuinely removed. The explanation may therefore reflect an effective model that differs from Φ [Yuan et al., 2023]; Agarwal et al. [2022] show this introduces a bounded but non-zero faithfulness gap. Second, each GNNEXPLAINER call is an independent optimization with no principled way to set $\alpha_1, \ldots, \alpha_4$ (Eq. 12), making it less stable across structurally identical nodes than amortized methods like PGExplainer [Luo et al., 2020].

Third, Faber et al. [2021] and Agarwal et al. [2022] argue that comparing an explanation to ground truth can itself be misleading: a badly-trained GNN may not use the planted motif at all, so a low score reflects the classifier, not the explainer. The clique GCN of Section 5 reaches a comparatively balanced 76.7% accuracy on clique nodes and 79.5% on base nodes, giving the explainer a genuine gradient signal on both classes, but some of the residual F1 gap may still not be GNNEXPLAINER's fault. The same applies even more sharply to our anchor-feature check (Section 6.2): since GNNEXPLAINER only reports what Φ already relies on (Section 4.8), any non-zero color weight we measure is a statement about Φ's training, not GNNEXPLAINER. We report it anyway, for simplicity and because evaluating faithfulness metrics is not this report's focus.

8 Conclusion and Future Work

We have presented a detailed study of GNNEXPLAINER. Along the way we identified a genuine error in the original derivation (Section 4.5): the step that replaces an intractable expectation over subgraphs with a single soft-masked forward pass invokes Jensen's inequality in the wrong direction, so the continuous relaxation underlying GNNEXPLAINER is best understood as a heuristic, not a provable bound. We then concretely investigate the local-optima behavior the original authors already anticipate but do not characterize, confirmed on the clique benchmark: F1 stays between 0.13 and 0.26 across clique sizes, worst at $k = 5$, and the optimizer never recovers a full clique. Then we adapt a 2026 unfaithfulness result for self-explainable GNNs into a testable question for GNNEXPLAINER: once the anchor features are made truly constant, its feature mask still cannot reproduce the clean, deterministic failure of a jointly-trained extractor, but one of the two anchor features receives a large, class-dependent weight gap out of proportion to the underlying classifier's accuracy gap.

Future work could address the mistake found in the derivation, it might be interesting to characterize its badness or to even investigate the false convexity assumption in more detail, perhaps considering whether there are distinguishable classes where this derivation works provably, or in general investigate which GNNs would fulfill the convexity assumption.

References

Chirag Agarwal, Marinka Zitnik, and Himabindu Lakkaraju. Probing GNN explainers: A rigorous theoretical and empirical analysis of GNN explanation methods. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2022.

Steve Azzolin, Stefano Teso, Bruno Lepri, Andrea Passerini, and Sagar Malhotra. GNN explanations that do not explain and how to find them. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2601.20815.

P. Erdős and A. Rényi. On random graphs I. *Publicationes Mathematicae Debrecen*, 6:290–297, 1959.

Lukas Faber, Amin K. Moghaddam, and Roger Wattenhofer. When comparing to ground truth is wrong: On evaluating GNN explanation methods. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2021.

Matthias Fey and Jan E. Lenssen. Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *International Conference on Machine Learning (ICML)*, 2017.

Sarthak Jain and Byron C. Wallace. Attention is not explanation. In *Proceedings of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2019.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.

Xiaoxiao Li and Joao Saude. Explain graph neural networks to understand weighted graph features in node classification. In *Discovery Science*, 2020. arXiv:2002.00514.

Zachary C. Lipton. The mythos of model interpretability. *Communications of the ACM (Queue)*, 16 (3), 2018.

Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

Dongsheng Luo, Wei Cheng, Dongkuan Xu, Wenchao Yu, Bo Zong, Haifeng Chen, and Xiang Zhang. Parameterized explainer for graph neural network. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020.

Siqi Miao, Miaoyuan Liu, and Pan Li. Interpretable and generalizable graph learning via stochastic attention mechanism. In *International Conference on Machine Learning (ICML)*, 2022.

Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. "why should I trust you?": Explaining the predictions of any classifier. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2016.

Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.

Rex Ying, Dylan Bourgeois, Jiaxuan You, Marinka Zitnik, and Jure Leskovec. GNNExplainer: Generating explanations for graph neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.

Hao Yuan, Jiliang Tang, Xia Hu, and Shuiwang Ji. XGNN: Towards model-level explanations of graph neural networks. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2020.

Hao Yuan, Haiyang Yu, Jie Wang, Kang Li, and Shuiwang Ji. On explainability of graph neural networks via subgraph explorations. In *International Conference on Machine Learning (ICML)*, 2021.

Hao Yuan, Haiyang Yu, Shurui Gui, and Shuiwang Ji. Explainability in graph neural networks: A taxonomic survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(5): 5782–5799, 2023. doi: 10.1109/TPAMI.2022.3204236.

A Implementation Details

All experiments use Python 3.13, PyTorch 2.12, and PyTorch Geometric 2.8 [Fey and Lenssen, 2019]. The full source code is provided as `gnnexplainer.py` and available at `https://github.com/ArionScheid/Seminar-in-AI.git`. Random seeds are fixed throughout for reproducibility. The GNNEXPLAINER implementation in PyTorch Geometric requires specifying the explanation type (`'model'`), mask types (`edge_mask_type='object'` and `node_mask_type='attributes'`), and model configuration. Using `mode='multiclass_classification'` with two output classes handles binary classification without requiring changes to the GCN output layer. As verified against the library source (`torch_geometric/explain/algorithm/gnn_explainer.py`), the `'attributes'` mask type creates one learnable weight per node *and* feature (Eq. 11), which is what we read out per node to obtain the color-anchor weights of Section 6.2.

B Feature-Mask Reparametrization: The Original Proposal

Ying et al. [2019] address the near-zero-value blind spot of Section 4 by not multiplying discarded feature entries by zero (or a value near it) at all. Instead, they treat the masked feature as a random variable and replace discarded entries with samples from that feature's own empirical marginal distribution, so a discarded feature is swapped for a *typical* value rather than suppressed toward zero. This avoids the confound where "small mask weight" and "small feature value" look identical to the optimizer. Concretely, using a reparametrization trick to keep the sampling step differentiable,

$$\mathbf{x}_j^F = \mathbf{z} + (\mathbf{x}_j - \mathbf{z}) \odot F, \qquad \textstyle\sum_k F_k \le K_F, \tag{14}$$

where $\mathbf{z}$ is sampled from the empirical marginal distribution of the node features and K_F caps the number of features kept in the explanation; the expectation over $\mathbf{z}$ is estimated by Monte Carlo sampling during training. Eq. (14) is the mechanism referenced in Section 4: PyTorch Geometric's GNNEXPLAINER implementation does not carry it out, using the plain sigmoid mask of Eq. (11) instead.

C The RBGV Example

Azzolin et al. [2026] illustrate the anchor-set construction of Section 2 with a synthetic graph-classification dataset, RBGV: every graph contains red and blue nodes, randomly connected, whose relative counts (#blue vs. #red) determine a binary label, plus one isolated green node and one isolated violet node present in *every* instance, forming an anchor set. An SE-GNN can reach perfect task accuracy while reporting only the anchor nodes as its explanation: let the extractor output the green node when reds dominate and the violet node otherwise,

$$e(G) = \begin{cases} u_{\text{green}} & \text{if } \#\text{red} \ge \#\text{blue} \\ u_{\text{violet}} & \text{if } \#\text{blue} > \#\text{red}, \end{cases} \qquad g(R) = \begin{cases} 0 & \text{if } R = u_{\text{green}} \\ 1 & \text{if } R = u_{\text{violet}}. \end{cases} \tag{15}$$

The composition $f = g \circ e$ is perfectly accurate, yet its reported explanation is color-blind to red and blue, the only nodes the label actually depends on. This is because e must internally count red and blue nodes to decide which anchor to report, but never reveals that computation. This is the template Section 4.8 adapts to our clique dataset.